

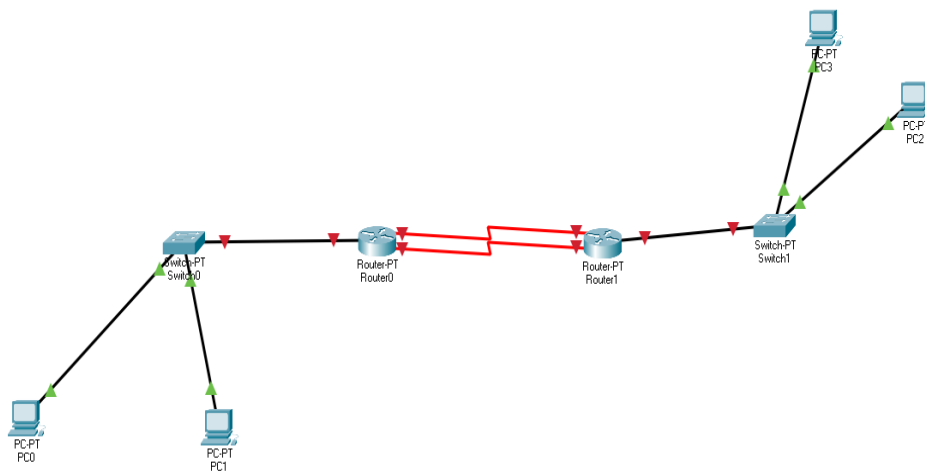
Lab Practical #14:

To study load balancer & reverse proxy in different network scenario.

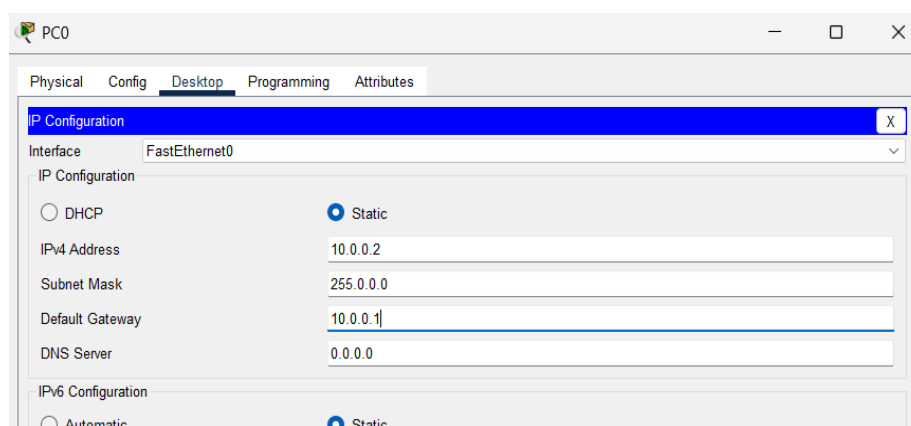
Practical Assignment #14:

A. To configure load-balancing with static route using packet tracer.

- Step 1 — Place devices
 - Open Cisco Packet Tracer.
 - 2. Add routers and PCs as shown in your topology.
 - 3. Connect PC0, PC1 to Switch0 -> R1 and PC2, PC3 to switch -> the destination router.
 - 4. Create two paths between the routers.

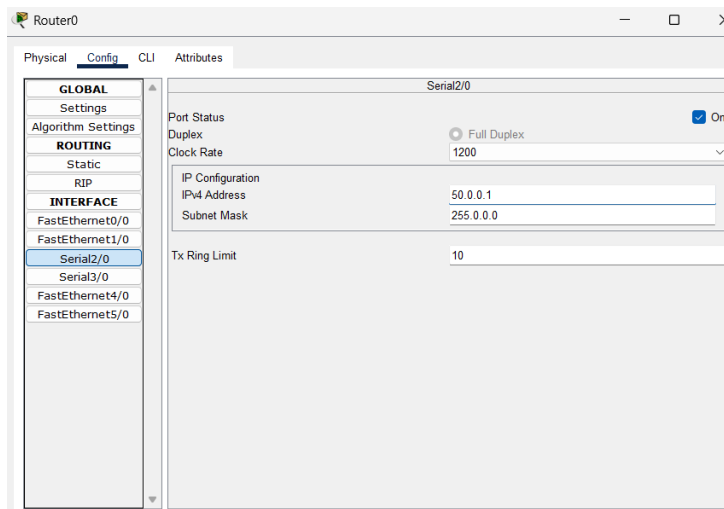


- Step 2 — Configure PC addresses
 - PC0: IP 10.0.0.2 | Mask 255.0.0.0 | Default Gateway 10.0.0.1
 - PC1: IP 10.0.0.3 | Mask 255.0.0.0 | Default Gateway 10.0.0.1
 - PC2: IP 20.0.0.2 | Mask 255.0.0.0 | Default Gateway 20.0.0.1
 - PC3: IP 20.0.0.3 | Mask 255.0.0.0 | Default Gateway 20.0.0.1

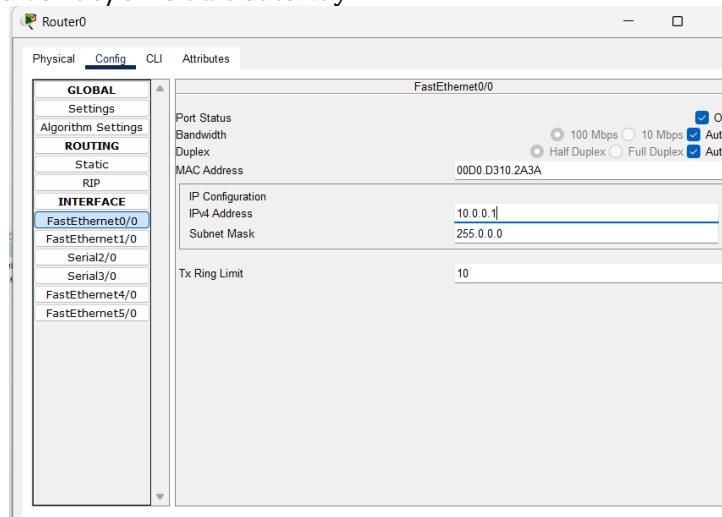


Date: / /

- Step 3 — Configure router interfaces
 - On each router, configure the required interface IP addresses, then use:
show ip interface brief
Router0:
 - on Serial2/0: 50.0.0.1
on Serial3/0: 60.0.0.1
 - Router1:
 - on Serial2/0: 50.0.0.2
on Serial3/0: 60.0.0.2

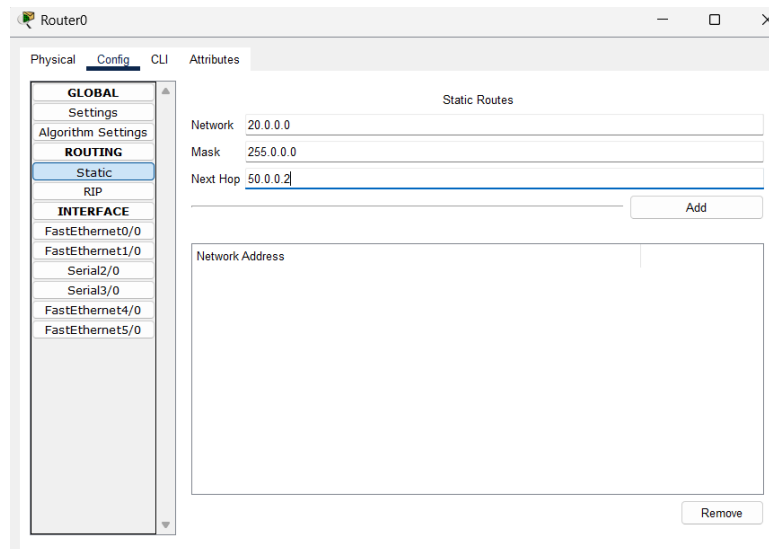


- Give Fa0/0 Default Gateway IP



- Step 4 — Configure static routes
 - On R1, add the two routes to the destination network using the correct reachable next-hop addresses:
ip route 20.0.0.0 255.0.0.0 <next-hop-1>
ip route 20.0.0.0 255.0.0.0 <next-hop-2>

Date: / /

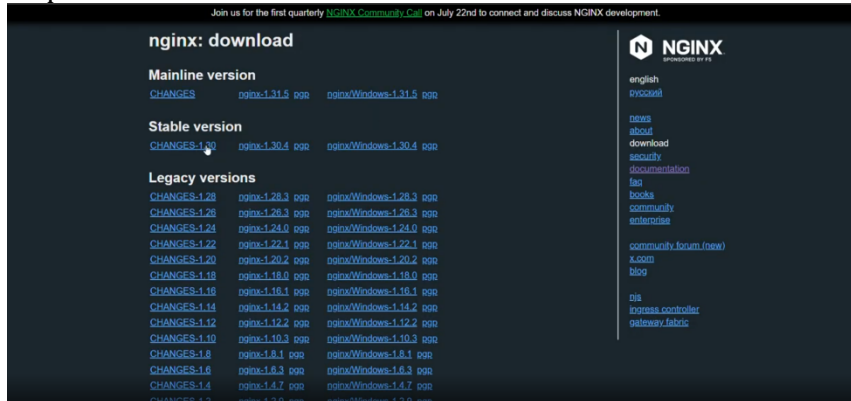


- Step 5 — Configure the return route
 - a route back to 10.0.0.0/24 on the destination router and any intermediate routers.
- Step 6 — Check the routes
 - show ip route
 - show ip route static
- Step 7 — Test
 - From PC1, run:
ping 20.0.0.2 or 20.0.0.3

Date: / /

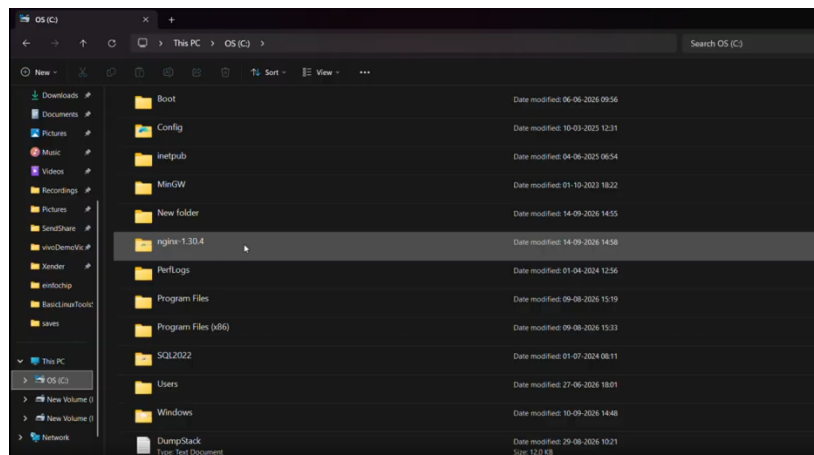
B. To configure reverse proxy using nginx.

- Step 1 — Download and install NGINX



1. Extract it, for example:

C:\nginx



2. Open **Command Prompt as Administrator**.

Go to the NGINX folder:

cd C:\nginx

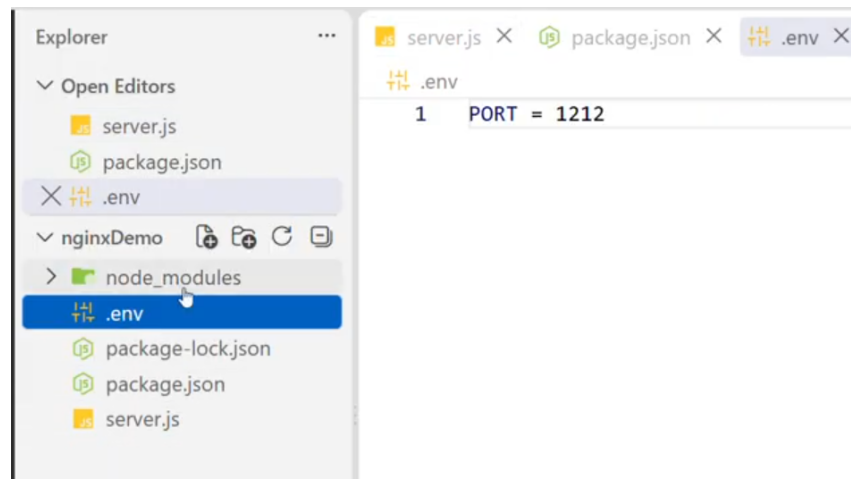


Date: / /

- Step 2 — Test the backend
 1. Make backend
 - a. Step 1:npm init -y (create package.json file)
 - b. Step 2:npm i express
 - c. Step 3:
Create Server using node.js
 - d. Server.js

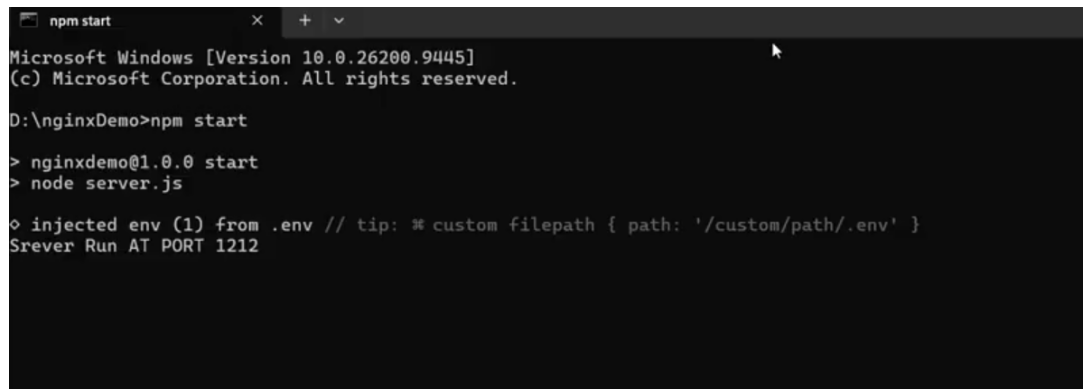


```
server.js X package.json 1 X .env X  
server.js  
1 const express = require('express');  
2 require('dotenv').config()  
3 const app = express()  
4 const PORT = process.env.PORT  
5  
6 app.get('/', (req, res) => {  
7   return res.send(`Server is Running on PORT ${PORT}`)  
8 })  
9 app.listen(PORT, () => {  
10   console.log(`Server Run AT PORT ${PORT}`)  
11 })
```



2. Run: curl http://localhost:1212 OR

Date: / /



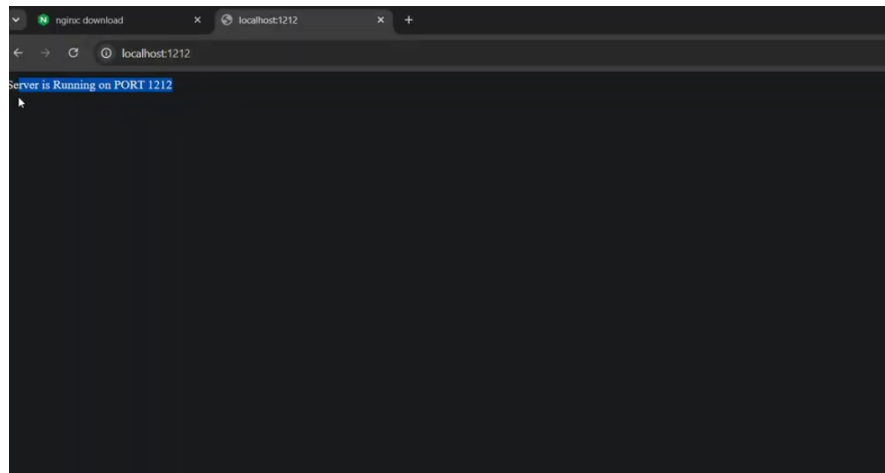
```
npm start
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

D:\nginxDemo>npm start

> nginxdemo@1.0.0 start
> node server.js

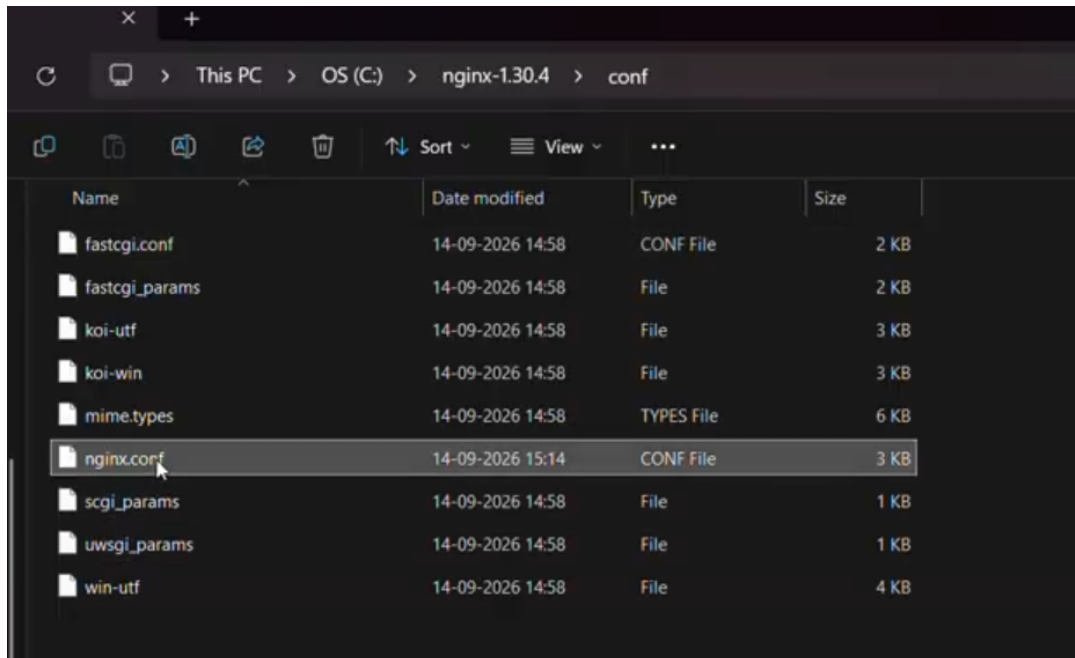
injectd env (1) from .env // tip: % custom filepath { path: '/custom/path/.env' }
Server Run AT PORT 1212
```

3. If the backend is working, you should receive its response.

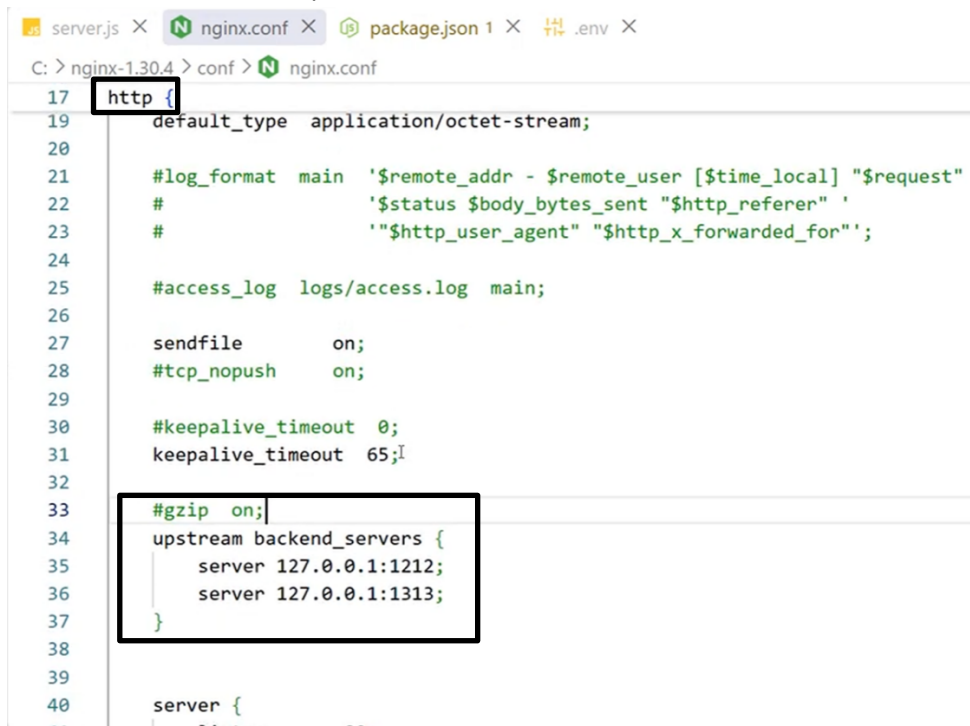


Date: / /

- Step 3 — Create the NGINX configuration
 1. Open: notepad C:\nginx\conf\nginx.conf



2. Inside the http { } section, add:



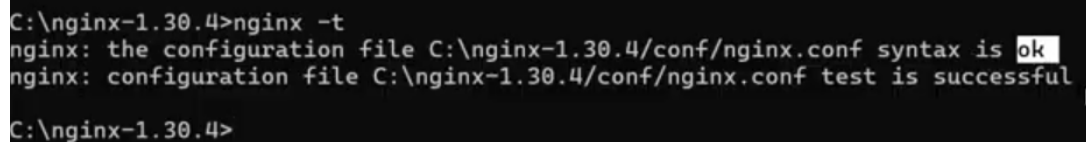
```
server.js X nginx.conf X package.json 1 X .env X
C: > nginx-1.30.4 > conf > nginx.conf
17 http {
18     default_type application/octet-stream;
19
20     #log_format main '$remote_addr - $remote_user [$time_local] "$request"
21     #                    '$status $body_bytes_sent "$http_referer" '
22     #                    '"$http_user_agent" "$http_x_forwarded_for"';
23
24     #access_log logs/access.log main;
25
26     sendfile on;
27     #tcp_nopush on;
28
29     #keepalive_timeout 0;
30     keepalive_timeout 65;
31
32
33     #gzip on;
34     upstream backend_servers {
35         server 127.0.0.1:1212;
36         server 127.0.0.1:1313;
37     }
38
39
40     server {
41         listen 80;
```

Important: Add only “server 127.0.0.1:1212” HERE

Date: / /

Important: Don't create a separate Linux-style /etc/nginx/sites-available/ folder unless you have specifically configured NGINX that way. The simplest Windows setup is to edit C:\nginx\conf\nginx.conf.

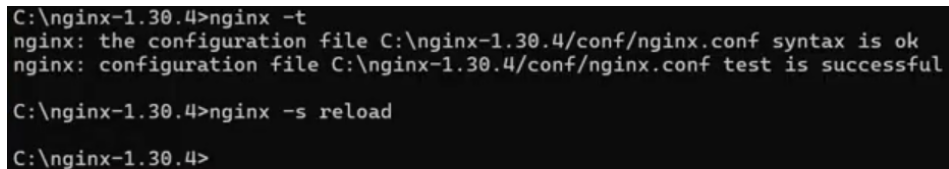
- Step 4 — Check NGINX configuration
 1. In Command Prompt:
 2. cd C:\nginx
 - a. Run: nginx.exe -t



```
C:\nginx-1.30.4>nginx -t
nginx: the configuration file C:\nginx-1.30.4/conf/nginx.conf syntax is ok
nginx: configuration file C:\nginx-1.30.4/conf/nginx.conf test is successful
C:\nginx-1.30.4>
```

3. You should see:
 - a. syntax is ok
 - b. test is successful

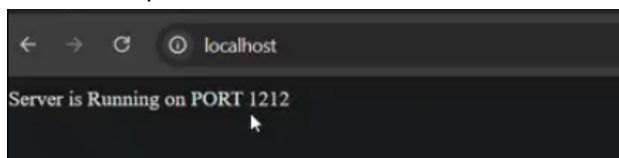
- Step 5 — Start NGINX
 1. If NGINX is already running, don't run it again. Use:
 - a. nginx.exe -s reload



```
C:\nginx-1.30.4>nginx -t
nginx: the configuration file C:\nginx-1.30.4/conf/nginx.conf syntax is ok
nginx: configuration file C:\nginx-1.30.4/conf/nginx.conf test is successful

C:\nginx-1.30.4>nginx -s reload
C:\nginx-1.30.4>
```

- Step 6 — Test the reverse proxy
 1. Open this in a browser: http://localhost /
 2. The response should come from the backend server:



Date: / /

C. To configure load balancer using nginx.

- Step 1 — Prepare backend servers

Create at least two backend servers. Give each a different response, for example:

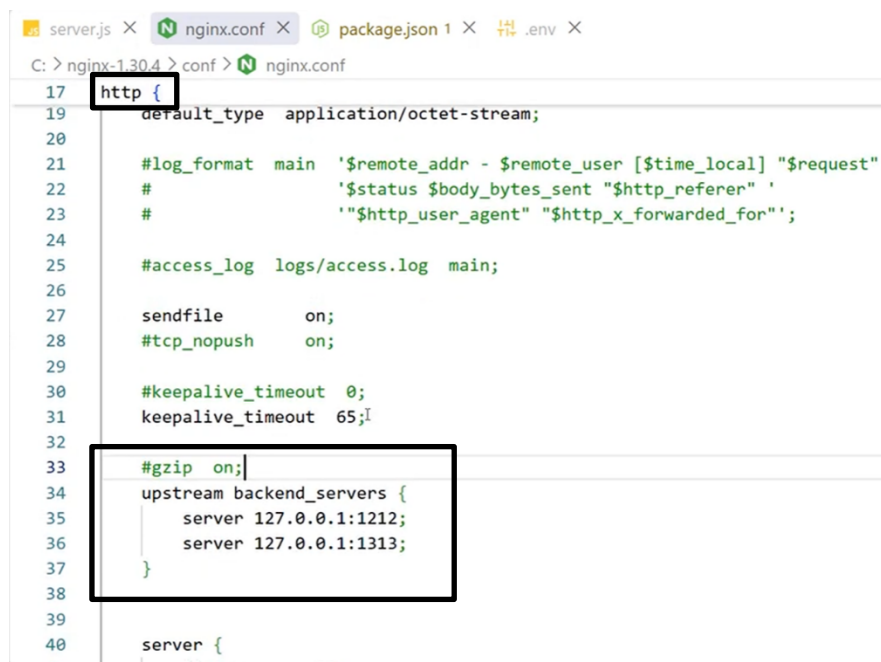
Server 1: Hello from SERVER 1

Server 2: Hello from SERVER 2

Just like B Step-2

- Step 2 — Test each backend
 1. Open this in a browser:
http://localhost /1212
http://localhost /1313
- Step 3 — Create the load balancer configuration
 1. go to C://nginx/nginx.confing
 2. Inside the http { } section, add:
upstream backend_servers {

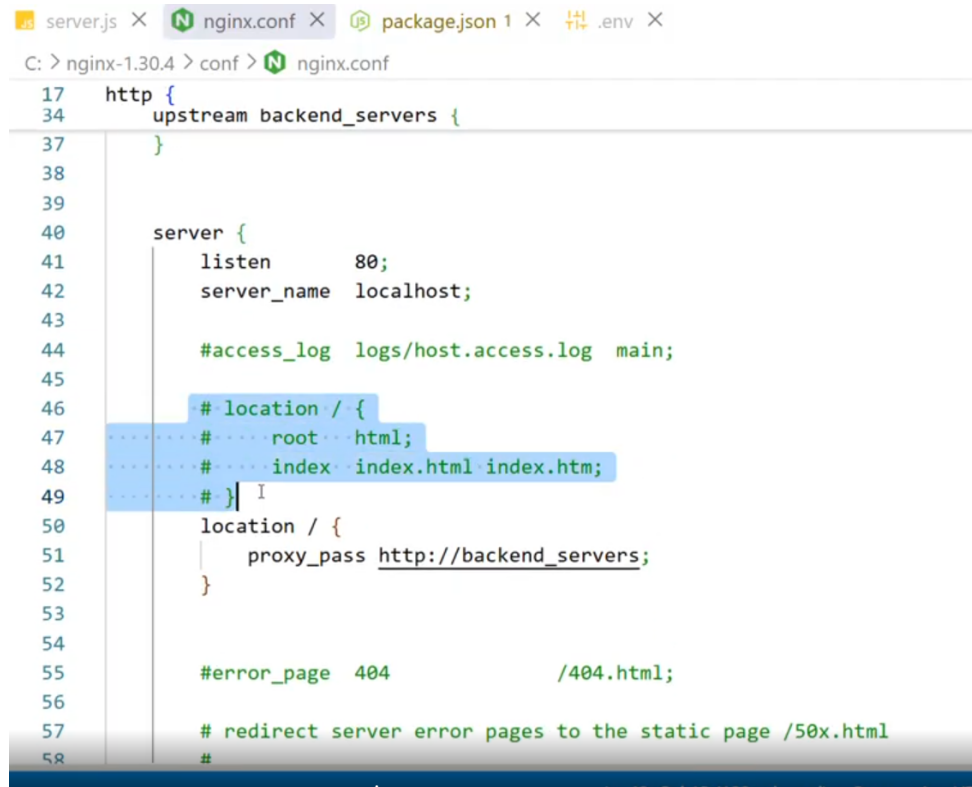
server 127.0.0.1:1212;
server 127.0.0.1:1313;
}



```
server.js x nginx.conf x package.json 1 x .env x
C: > nginx-1.30.4 > conf > nginx.conf
17 http {
19     default_type application/octet-stream;
20
21     #log_format main '$remote_addr - $remote_user [$time_local] "$request"
22     #                    '$status $body_bytes_sent "$http_referer" '
23     #                    '"$http_user_agent" "$http_x_forwarded_for"';
24
25     #access_log logs/access.log main;
26
27     sendfile on;
28     #tcp_nopush on;
29
30     #keepalive_timeout 0;
31     keepalive_timeout 65;
32
33     #gzip on;
34     upstream backend_servers {
35         server 127.0.0.1:1212;
36         server 127.0.0.1:1313;
37     }
38
39
40     server {
41         listen 80;
```

Date: / /

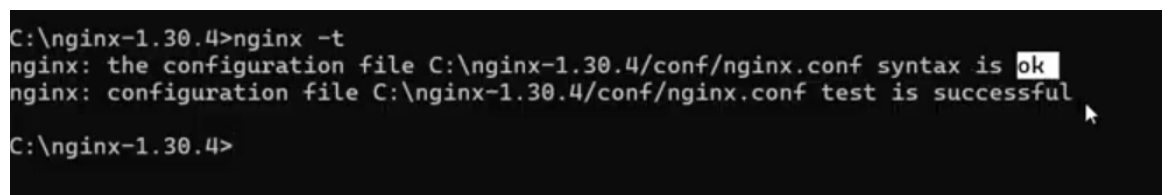
3. Inside the server {} section, add:



```
17 http {
34     upstream backend_servers {
37     }
38
39
40     server {
41         listen      80;
42         server_name localhost;
43
44         #access_log logs/host.access.log main;
45
46         # location / {
47         #     root html;
48         #     index index.html index.htm;
49         # }
50         location / {
51             proxy_pass http://backend_servers;
52         }
53
54
55         #error_page 404              /404.html;
56
57         # redirect server error pages to the static page /50x.html
58         #
```

Important: Comment existing location{} and add above location{} code

- Step 4 — Check NGINX configuration
 1. In Command Prompt:
 - a. cd C:\nginx
 - b. nginx.exe -t



```
C:\nginx-1.30.4>nginx -t
nginx: the configuration file C:\nginx-1.30.4/conf/nginx.conf syntax is ok
nginx: configuration file C:\nginx-1.30.4/conf/nginx.conf test is successful
C:\nginx-1.30.4>
```

2. You should see:
 - a. syntax is ok
 - b. test is successful

Date: / /

- Step 5 — Start NGINX
If NGINX is already running, don't run it again. Use:
nginx.exe -s reload

```
C:\nginx-1.30.4>nginx -t
nginx: the configuration file C:\nginx-1.30.4/conf/nginx.conf syntax is ok
nginx: configuration file C:\nginx-1.30.4/conf/nginx.conf test is successful

C:\nginx-1.30.4>nginx -s reload

C:\nginx-1.30.4>
```

Step 6 — Test Load Balancer in Browser

1. Open a web browser.
2. Enter:
http://localhost /
3. Refresh the page several times using F5.
4. Check whether the response changes between Port 1212 and Port 1313.
5. Different responses confirm that NGINX is distributing requests between the backend servers.